

Python for Research

Why this language?

Lecturer: Dr Tom Crossland

Slides by: Tom Crossland

The default language for research

Why?

The default language for research

Why?

- Ease of use
- Multiple supported paradigms
- Large ecosystem



Why *not* Python?

Why *not* Python?

- Slow?
- Dynamic typing?
- Difficulty of use



- Python is a *dynamically typed* programming language
 - ▶ The *type* of data is determined at runtime, when it is used
- This makes Python very flexible
 - ▶ It is easy to write and iterate Python programs quickly

- Python is a *dynamically typed* programming language

- Python is a *dynamically typed* programming language
- A *statically typed* programming language assigns a type to a variable when it is declared
 - ▶ It is easier to make guarantees about statically typed programs
 - ▶ Automatic tools can find problems before your code is run
- In Python, you may only find the problem when the code runs *that particular line*

- Python is an *interpreted* programming language
- The code is run line-by-line, one at a time
- This means Python works well at the REPL and in notebooks
 - ▶ (REPL, Read-Evaluate-Print-Loop, pronounced “REP-ul”)
 - ▶ (You can write Python code directly into the terminal)
 - ▶ (Many languages can be used as a REPL)
- Again, very flexible, easy to iterate

- Python is an *interpreted* programming language

- Python is an *interpreted* programming language
- Python code is not *compiled*
 - ▶ In many programming languages, code must first be converted into *machine code* that the computer understands
 - ▶ (Python *sort of* does this, with Just-In-Time (JIT) compilation)
- Compiled languages allow you to identify certain kinds of errors faster
- But this comes with much more complexity of implementation

Hello World

The “Hello World” program is the standard rite of passage when learning any new programming language

Here is Python's:

```
print('Hello World')
```

Hello World

Here is Hello World in C++:

```
#include<iostream>

using namespace std;

int main() {
    cout << "Hello World" << endl;
    return 0;
}
```

Hello World

And in Java:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

See why people like Python?

Speed



- So if it's not the fastest (by a long shot), why use it?

- So if it's not the fastest (by a long shot), why use it?
- Because Python can use libraries written in other languages
 - ▶ Numpy, C++ (fast CPU)
 - ▶ PyTorch, CUDA (GPU acceleration)
- This means we get all the benefits of these other languages
- But with Python syntax
 - ▶ Remember the C++/Java programs?

- Most importantly: Python has a *very developed* ecosystem
- This refers to the packages and tools written for the language
 - ▶ Getting towards 1 million packages available in the Python Package Index
- Anything you could want to do, there's a Python package

- Python has a very robust scientific ecosystem
- This is built on a foundation of Free and Open Source Software (FOSS)
- There are a number of fundamental libraries
 - ▶ NumPy
 - ▶ SciPy
 - ▶ scikit-learn
 - ▶ pandas
- And many specialised libraries that built on these

Scientific Python Ecosystem





Astrophysics



Biological
Computation



NiBabel

Neuro-Imaging
File Formats



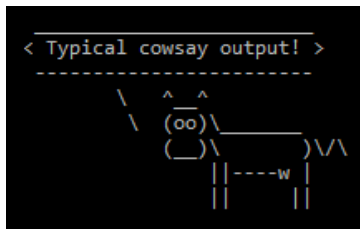
Mathematics
Software

Ecosystem

And, of course, all the other essentials:

```
>>> from coolname import generate_slug
>>> generate_slug()
'tiny-optimal-crocodile-of-progress'
>>> generate_slug()
'cherubic-leopard-of-delightful-serendipity'
>>> generate_slug()
'wealthy-athletic-swift-of-tempest'
```

```
>>> import cowsay
```



- So, where does our code actually run?

- So, where does our code actually run?
 - ▶ Locally (our own computer)
 - ▶ Remotely (a computer somewhere else)
 - ▶ An HPC cluster (High-Performance Computing)
 - ▶ Cloud computing
 - ▶ A specific machine owned by your lab
- All of these are viable options
- There are advantages and disadvantages to each

- It's also worth thinking about data storage
 - ▶ How much data do I have?
 - ▶ Are there restrictions on where it can be stored?
 - ▶ Do I need volume or latency?
 - ▶ What machines/people need access to it?
- These sorts of questions can inform your design decisions

- This is, mostly, about writing code which is:
 - ▶ Maintainable
 - ▶ Reliable
 - ▶ Scalable
- You should design code with these in mind

- To start with:
 - ▶ Good variable names
 - ▶ Good function names
 - ▶ Sensible modules
- More complex ideas:
 - ▶ Modularity
 - ▶ Separation of concerns
 - ▶ DRY (Don't Repeat Yourself)
 - ▶ Encapsulation

- We want to avoid:
 - ▶ Spaghetti code
 - ▶ Lasagne code
 - ▶ Ravioli code
 - ▶ Pizza code

- Good code is *testable*
- *Unit tests* are written to test individual components
- *Integration tests* are testing how components interact
- *End-to-End testing* (“E2E”) tests the whole piece of software

- Are we familiar with:
 - ▶ `final_version.doc`
 - ▶ `final_version2.doc`

- Are we familiar with:
 - ▶ `final_version.doc`
 - ▶ `final_version2.doc`
 - ▶ `final_final_version.doc`
 - ▶ `final_final_pleasegod_version15.doc`

- Are we familiar with:
 - ▶ `final_version.doc`
 - ▶ `final_version2.doc`
 - ▶ `final_final_version.doc`
 - ▶ `final_final_pleasegod_version15.doc`
- We can avoid this by using *version control* systems
- `git` is the industry standard
- It uses a series of individual *commits*, representing changes, and stores a history of them
- Very useful for going back to previous versions, or having multiple different versions in parallel